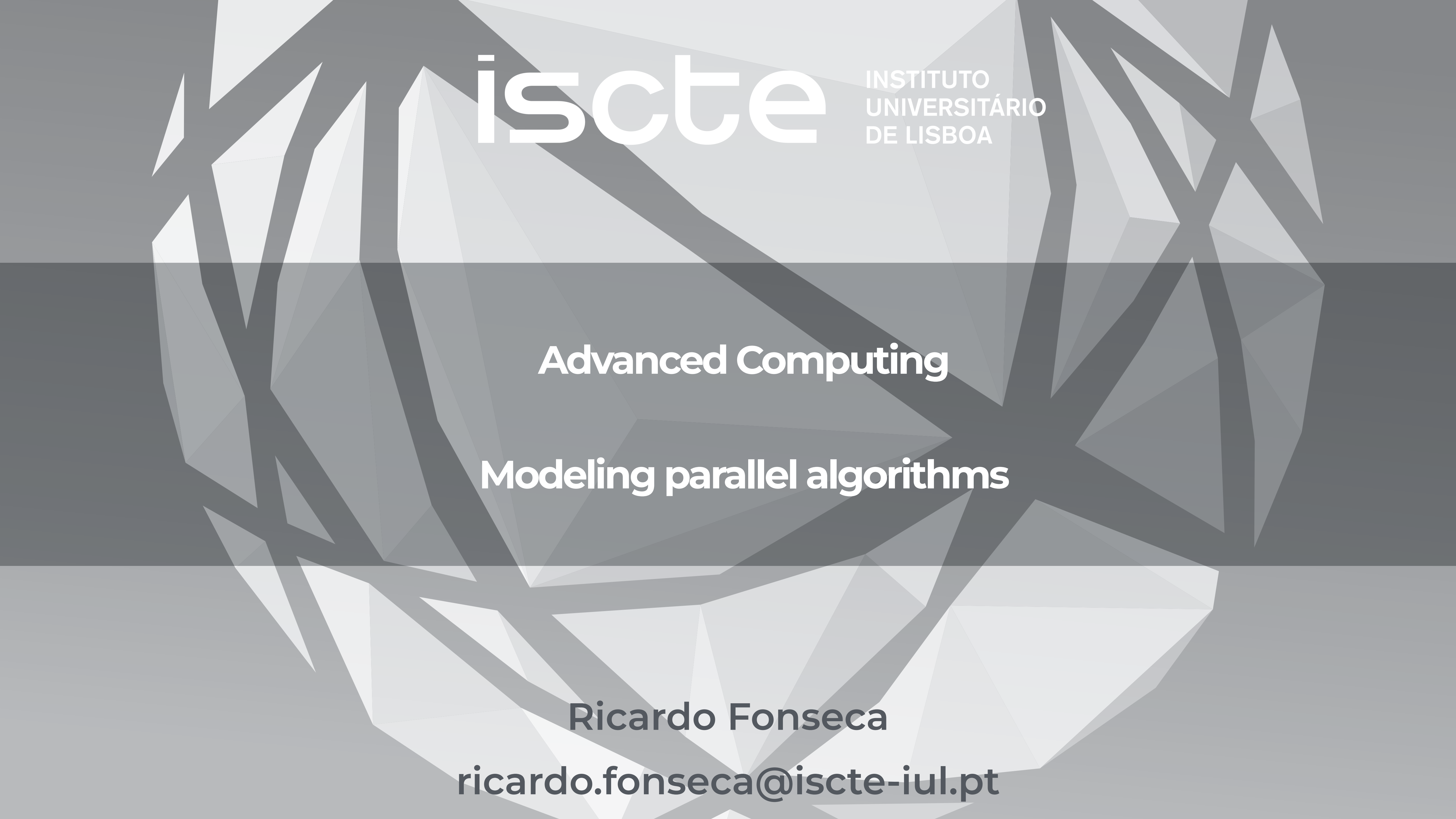




**iscte**

INSTITUTO  
UNIVERSITÁRIO  
DE LISBOA

**Advanced Computing**

**Modeling parallel algorithms**

**Ricardo Fonseca**

**[ricardo.fonseca@iscte-iul.pt](mailto:ricardo.fonseca@iscte-iul.pt)**

# Modeling parallel algorithms

- **Introduction**

- Why model parallel algorithms?
- Parallel overhead

- **Performance metrics**

- Speedup, efficiency and cost

- **Performance models**

- Roofline model
- Latency-bandwidth models
- Parallel load balance

- **Scaling laws**

- Strong and weak scaling
- Amdahl and Gustafson laws

- **Example**

- 2D stencil decomposition



**ILLIAC IV** at NASA Ames Research Center (1974)



# Introduction

# Why model parallel algorithms?

- **Parallel scaling is critical part of high-performance computing**
  - Performance modeling helps us understand why parallel programs scale, or why they fail to scale
  - A performance model is not meant to predict runtimes perfectly, it is mean to explain dominant costs and scaling behaviors
- **In HPC, we often ask**
  - Will my code run faster if I use more cores / nodes?
  - What limits performance?
  - Is the bottleneck computation, memory, communication, synchronization, or imbalance?
  - What should I optimize first?
  - How large a problem do I need to efficiently use a machine?

# A simple view of parallel runtime

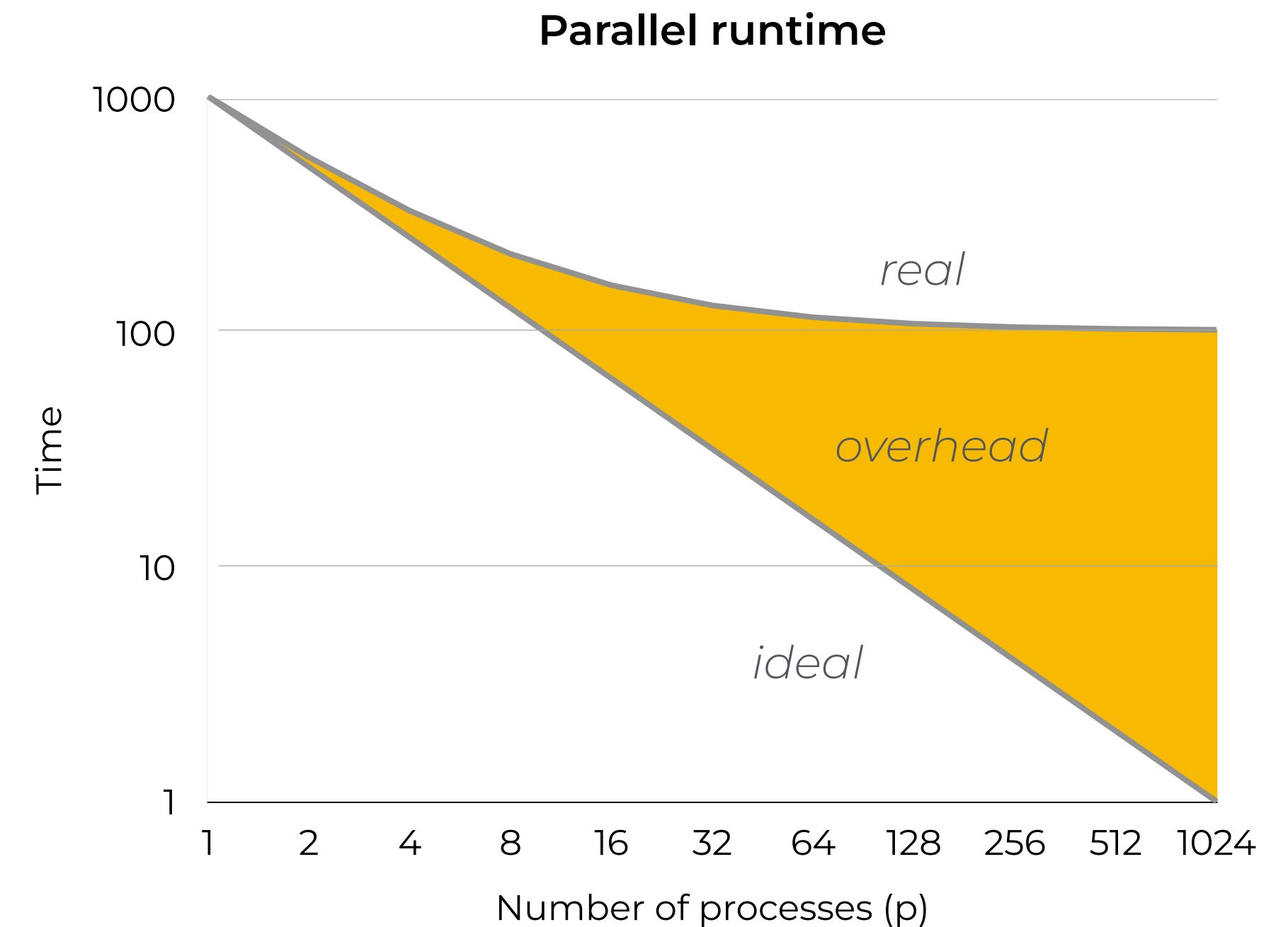
$$T_p = T_{\text{compute}} + T_{\text{memory}} + T_{\text{comm}} + T_{\text{sync}} + T_{\text{imbalance}} + T_{\text{overhead}}$$

- **The time spent by a parallel program depends on more than computation**
  - There are many additional terms contributing to the execution time
- **The runtime on  $p$  processors,  $T_p$ , will include**
  - $T_{\text{compute}}$  : useful computation work
  - $T_{\text{memory}}$  : data movement through the memory hierarchy (e.g. cache and NUMA effects)
  - $T_{\text{comm}}$  : data movement between processes / nodes (communication)
  - $T_{\text{sync}}$  : waiting at barriers, reductions (synchronization)
  - $T_{\text{imbalance}}$  : idle time cause by unequal work
  - $T_{\text{overhead}}$  : extra work introduced by the parallelization of the algorithm (e.g. the parallel algorithm requires more operations than the serial version)
- **More often than not, the additional terms will dominate runtime**
  - We refer to the sum of these terms as the “parallel overhead”



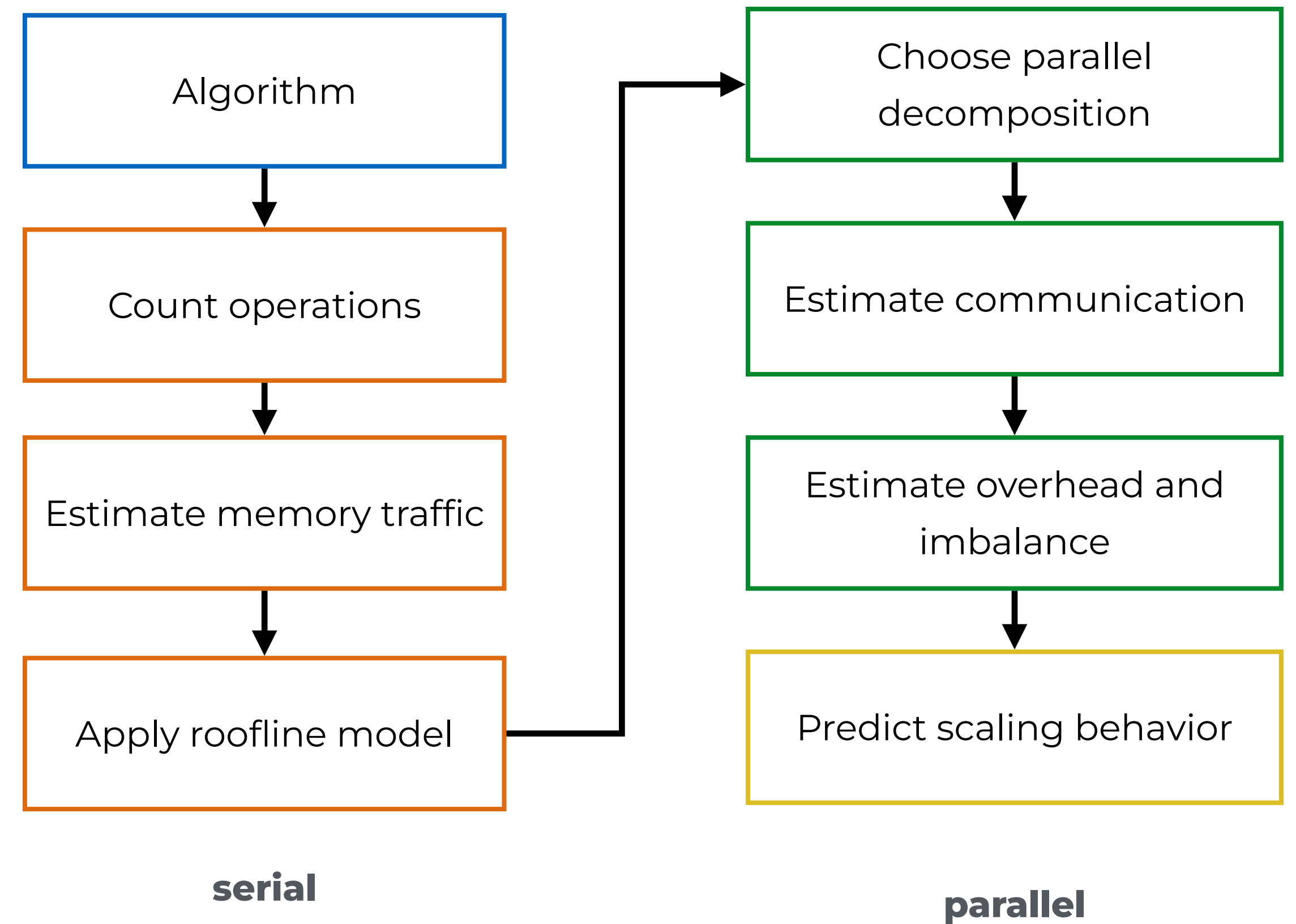
# Parallel overhead

- **The parallel overhead  $T_o$  is the extra cost introduced by parallel execution**
  - It is usually defined as the difference between an ideal parallel runtime and the real parallel runtime
  - As  $p$  increases we expect computation time to go down
  - However the time for communication / synchronization / imbalance may not decrease at the same time
  - At some point  $T_o \sim T_{\text{compute}}$  and adding more processes gives little benefit
- **A general model for parallel runtime would then be**
  - $T_p = \frac{W}{p} + T_o(p, N)$
  - Where  $W$  is the total useful work distributed over  $p$  processors, and
  - $T_o(p, N)$  is the overhead depending on both the number of processors and problem size



# Modeling workflow

- **The model for our parallel program should therefore account for more than computation**
  - It should account for single process performance and for parallel overhead
- **A good model should therefore combine**
  - Algorithmic work and memory behavior
  - Communication pattern
  - Parallel overhead
  - Scaling law



# Performance metrics



# Speedup and Efficiency

- **The two main metrics of parallel computing are speedup and efficiency**
  - Speedup measures how much faster you algorithm will run
  - Efficiency measures how well you are using your parallel hardware

- **Speedup**

- For  $p$  processes the speedup is defined as:

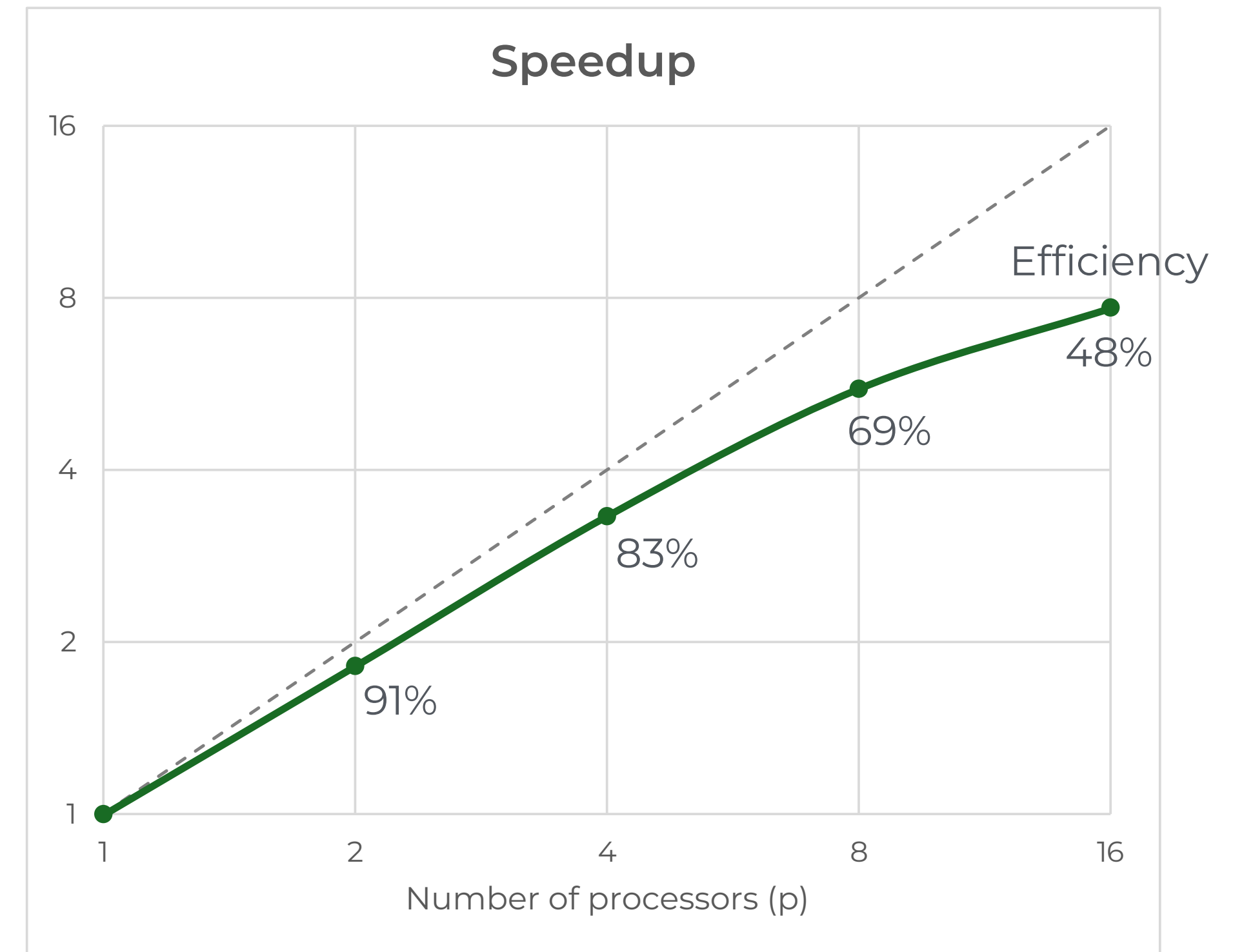
$$S_p = \frac{T_1}{T_p}$$

- Where  $T_1$  is the runtime on 1 processor and  $T_p$  is parallel runtime
  - Ideally (zero parallel overhead) we would have  $S_p = p$

- **Efficiency**

- Efficiency is defined as:

$$E_p = \frac{S_p}{p}$$



| Processors | Runtime | Speedup | Efficiency |
|------------|---------|---------|------------|
| 1          | 100 s   | 1.00    | 1.00       |
| 2          | 55 s    | 1.82    | 0.91       |
| 4          | 30 s    | 3.33    | 0.83       |
| 8          | 18 s    | 5.56    | 0.69       |
| 16         | 13 s    | 7.69    | 0.48       |

# Cost of a parallel computation

- **The computational cost measures the total processor-time used by the parallel algorithm**

- Computational cost is defined as

$$C_p = p \times T_p$$

- Usually measured in core hours
- Same cost for a program taking 10 hours on 10 processors, or 1 hour on 100 processors

- **This is usually how HPC workload are charged**

- User projects are allocated a number of core hours for parallel computing work

- **A parallel algorithm is said to be cost optimal if the cost is the same as for the serial version**

- That is  $C_p = p \times T_p = T_1$

- **Unfortunately, this would require perfect scaling efficiency**

- Parallel computations will generally cost more than the equivalent serial ones
- However, they are faster / larger than what is attainable on a single processor

# Performance models



# Serial performance

- **The baseline for parallel performance metrics is the performance of the serial version,  $T_1$** 
  - The performance of the serial version does not suffer from parallel overhead
- **The main parameters affecting serial runtime are compute capability and memory bandwidth**
  - More complex models may take into account other parameters, such as vectorization or memory hierarchies
- **To model serial performance we begin by defining Arithmetic Intensity**
  - Also referred to as operational or computational intensity
  - Arithmetic intensity is defined as the ratio between the number of operations (FLOP)  $W$ , and the memory traffic (number of bytes transferred)  $Q$

$$I = \frac{W}{Q} \text{ (FLOPS/byte)}$$

- **Usually algorithms are classified as**
  - High intensity (  $I > 100$  FLOPs / byte ) - e.g. Matrix multiplication, compute bound (limited by compute capability)
  - Low intensity (  $I < 100$  FLOPs / byte )- e.g. Vector addition, memory bound (limited by memory constraints)



# Roofline model

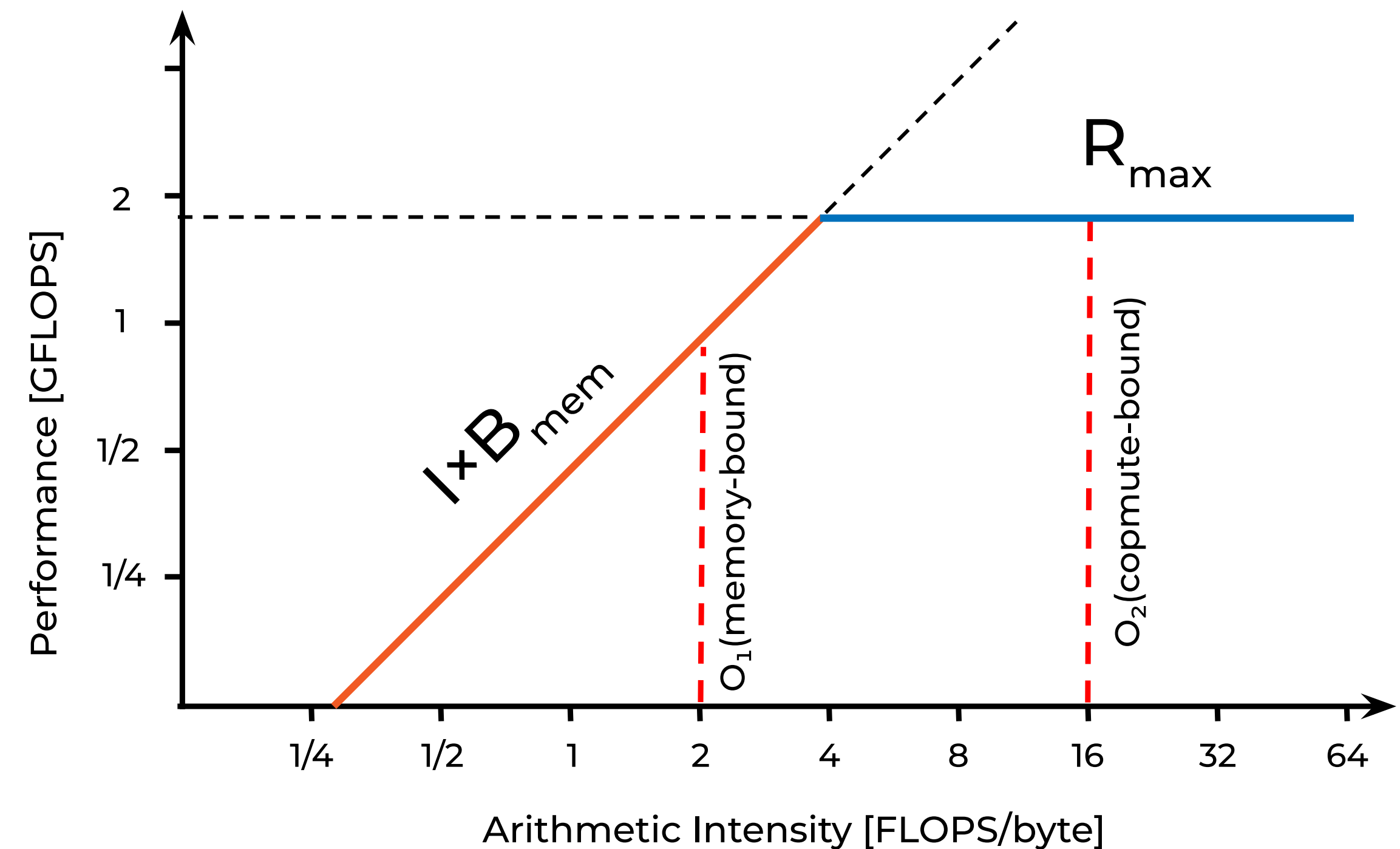
- **For a given system, peak performance will depend on the compute intensity**
  - For memory-bound problems, the performance will be limited by  $I \times B_{\text{mem}}$ , where  $B_{\text{mem}}$  is the peak memory bandwidth
  - For compute-bound problems, the performance will be limited by peak compute performance,  $R_{\text{max}}$

- **Change in behavior happens at the so-called ridge point,  $I^*$**

$$I^* = \frac{R_{\text{peak}}}{B_{\text{mem}}}$$

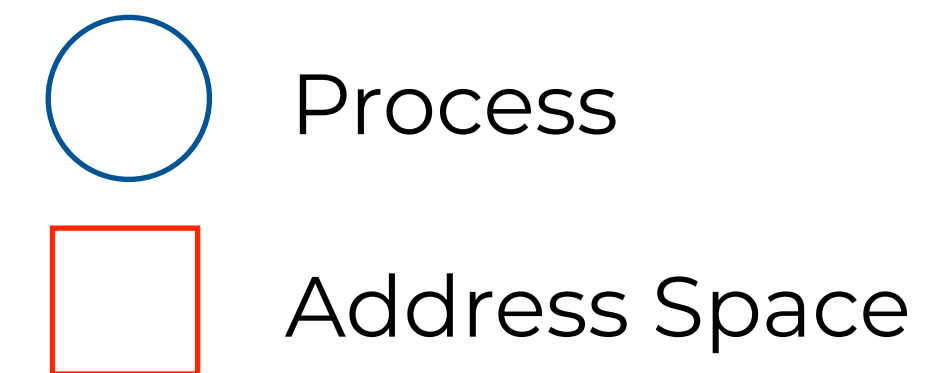
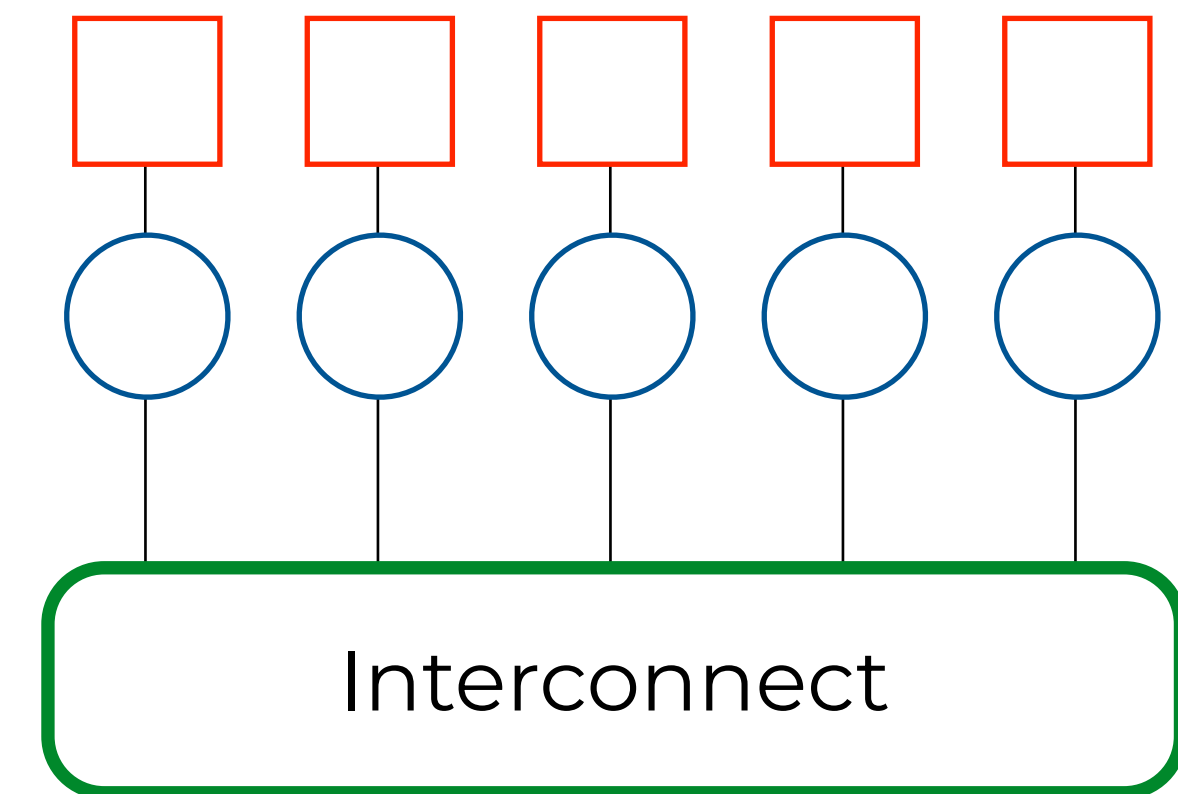
- If  $I < I^*$  the kernel is memory bound
- If  $I > I^*$  the kernel is compute bound
- **Real performance will always fall below the roofline limit**
  - Memory hierarchy and other hardware / program details are ignored

$$R = \min(R_{\text{max}}, I \times B_{\text{mem}})$$



# Communication and Network performance

- **In distributed-memory systems:**
  - Each process has private memory
  - Data is exchanged using messages
- **Examples:**
  - point-to-point messages, Halo exchanges
  - Reductions, Broadcasts
  - All-to-all communication
- **In modern HPC, moving data is often more expensive than computing**
  - Modeling interconnect is crucial to understanding parallel program performance



# Latency bandwidth model

- **Communication usually involves latency and bandwidth costs**

- The latency-bandwidth model estimates the time to send a message of size  $m$  as

$$T_{\text{comm}} = L + \frac{m}{B_{\text{net}}}$$

- Where  $L$  is the latency (or startup time),  $B_{\text{net}}$  is the peak network bandwidth (bytes per second), and  $n$  is the message size in bytes

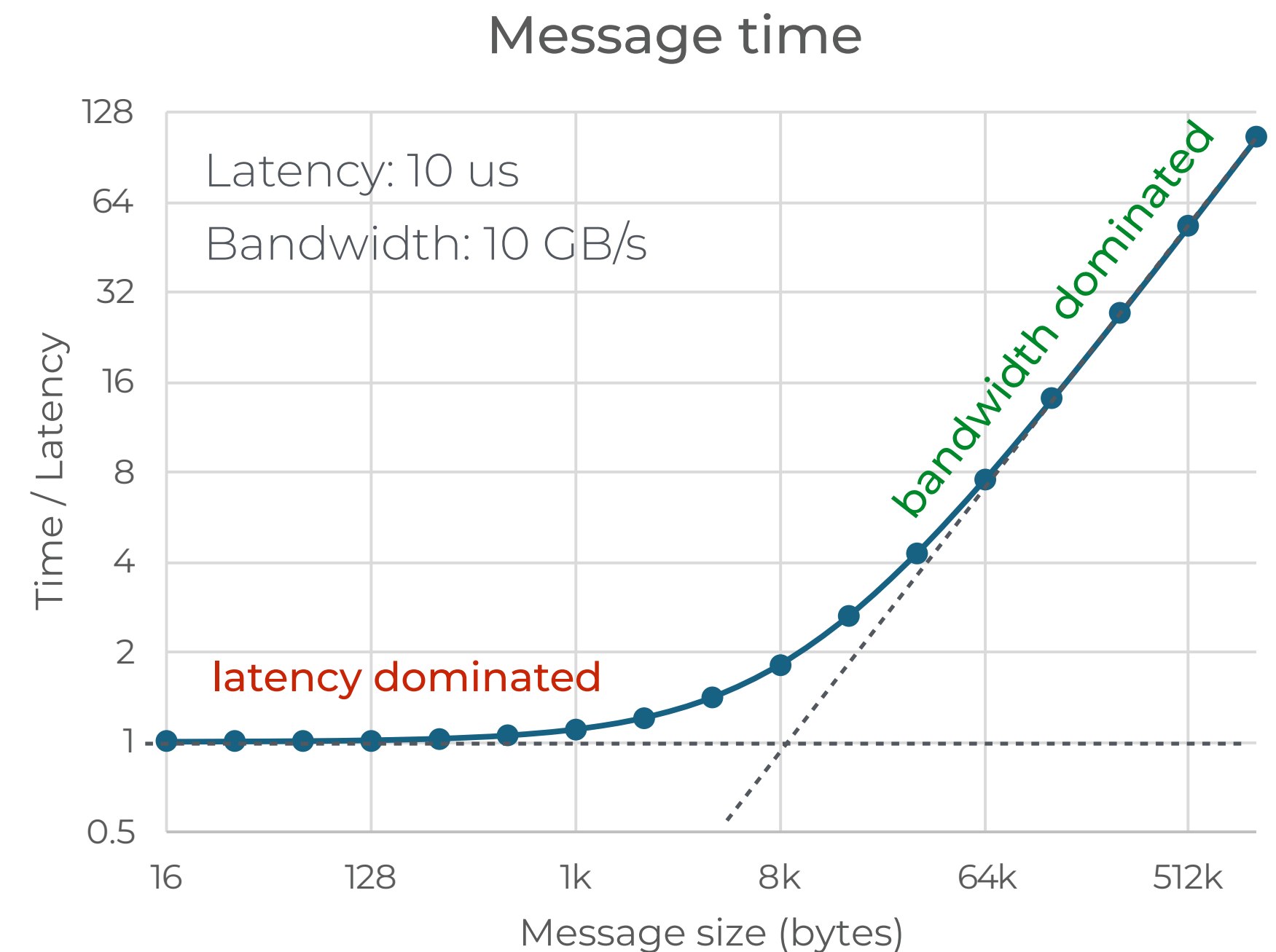
- **Behavior depends on message size**

- For small messages the latency term will dominate, i.e.,  $T_{\text{comm}} \simeq L$
- For large messages the bandwidth term will dominate, i.e.,  $T_{\text{comm}} \simeq m/B_{\text{net}}$

- **Sending  $n$  messages with total data volume  $M$  leads to**

$$T_{\text{comm}} = nL + \frac{M}{B}, \text{ where } M = \sum_{i=1}^n m_i$$

- For the same total volume, sending many small messages is more expensive because of the latency cost





# Collective communication models

- **Collective operations involve exchanging messages between multiple nodes**
  - Common operations include broadcast, reduce, all reduce, scatter, gather, all-to-all

- **Collective communications are implemented using a series of steps**
  - On each step, multiple processors exchange messages simultaneously
  - Communication time must also consider the number of message steps

- **For a collective communication involving  $m$  bytes**

- Approximate tree-based reduction model

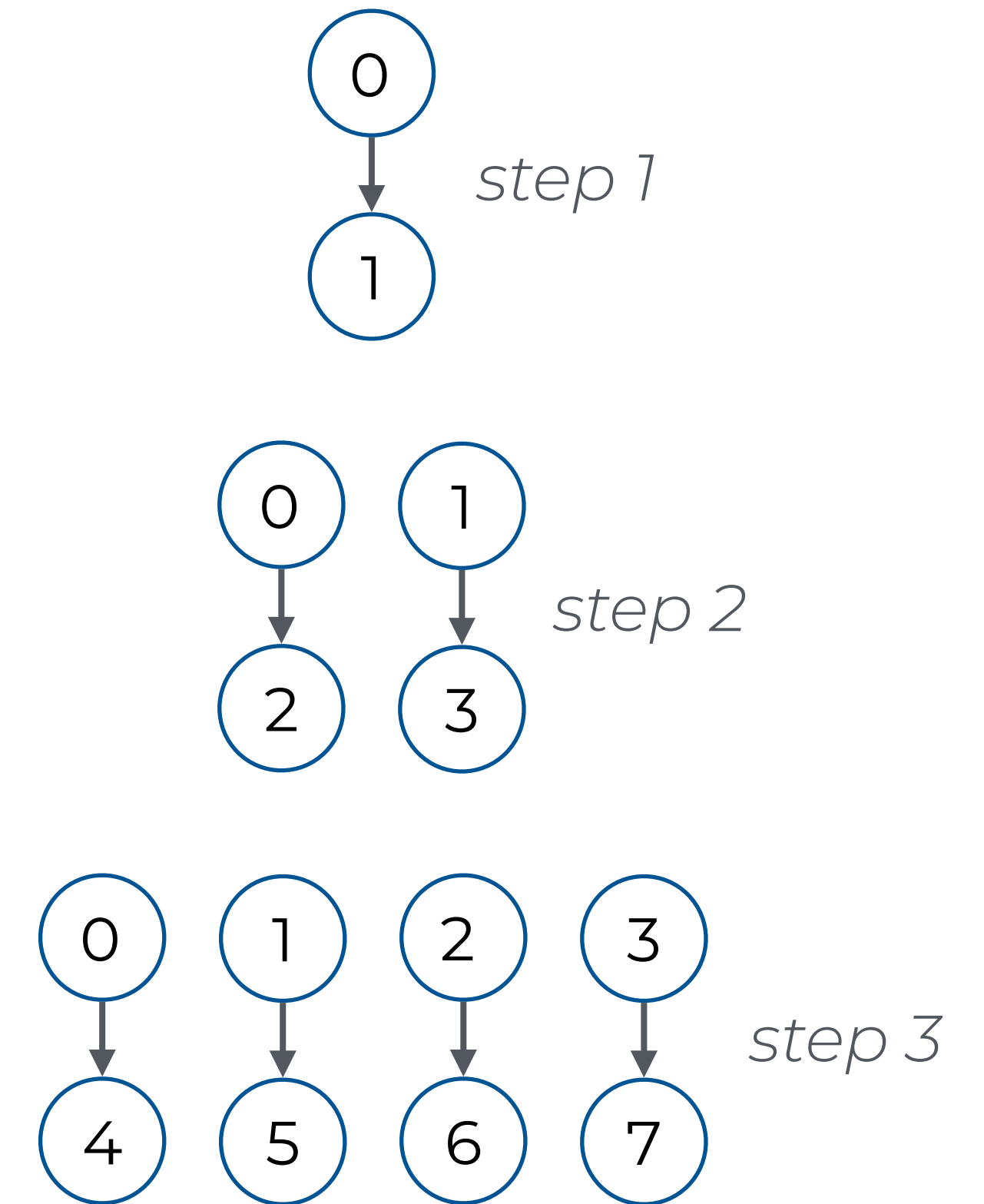
$$T_{\text{reduce}} = \mathcal{O}(\log p)(L + m/B)$$

- Approximate all-to-all model

$$T_{\text{all-to-all}} = \mathcal{O}(pL + pm/B)$$

- **Note that communication time increases with process count**

- Collectives may become scalability bottlenecks because many processes participate



*broadcast example*

# Parallel scalability and communications

- **Given the time spent in communications these will often dominate parallel scalability**
  - Communication structure can be more important than total data volume
  - Communication patterns matter: different patterns have different scalability
- **To estimate the impact of communication in parallel scalability we use the computation to communication ratio  $R$**

$$R = \frac{\text{computation}}{\text{communication}}$$

• Where we can use FLOPS/byte, i.e.,  $R = \frac{\text{FLOPs}}{\text{bytes communicated}}$ ,

• or time, i.e.,  $R = \frac{T_{\text{compute}}}{T_{\text{comm}}}$

- **The value  $R$  can be used to estimate parallel scalability limitations due to communication**
  - High values of  $R$  mean good data locality, more useful work per communicated byte, and better scalability
  - Low values of  $R$  mean that the algorithm is communication heavy, with limited scalability

| Pattern              | Example           | Scaling concern         |
|----------------------|-------------------|-------------------------|
| Nearest neighbor     | Stencils          | Usually scalable        |
| Tree communication   | Reductions        | Grows with $\log(p)$    |
| Broadcast            | Distributing data | Can bottleneck at scale |
| All-to-all           | FFT, transpose    | Often expensive         |
| Random communication | Graph algorithms  | Hard to optimize        |



# Load balance

- **Good parallel scalability requires all processors to be active**

- The work must be evenly distributed across processor for the be achieved
- If processor  $i$  has work time  $T_i$ , the the parallel step time will be

$$T_p = \max_i (T_i)$$

- The fastest processors will have to wait for the slowest ones

- **We use the term “load balance” to describe how evenly work is distributed across processors**

- A simple load balance metric is the ration between average processor runtime and maximum processor runtime

$$LB = \frac{\bar{T}}{T_p}$$

- where  $\bar{T} = \frac{1}{p} \sum_{i=1}^p T_i$  is the average processor runtime

- **A perfectly load balanced situation has LB = 1**

- Whenever  $LB < 1$ , there is imbalance, and parallel scalability suffers

| Process | Time        |
|---------|-------------|
| 0       | 10 s        |
| 1       | 11 s        |
| 2       | 9 s         |
| 3       | <b>20 s</b> |

## Example

- Average time  $\bar{T} = 12.5$  s
- Maximum time  $T_3 = 20$  s
- Load balance  $LB = 12.5/20 = 0.625$

# Ensuring good load balance

- **Load imbalance may cause significant delays in parallel program execution**
  - Common causes are unequal domain sizes, particle clustering, dynamic workloads, and operating system noise
  - Parallel algorithms must carefully optimize load balance, either statically or dynamically
- **Static load balancing**
  - Work assigned before execution
  - Low scheduling overhead, Works well for predictable workloads
  - **Examples:** Regular grids, Dense linear algebra, Uniform particle distributions
- **Dynamic load balancing**
  - Work assigned or adjusted during execution
  - Better for irregular workloads, Adds scheduling and migration overhead
  - **Examples:** Task queues, dynamic domain repartitioning, adaptive mesh refinement

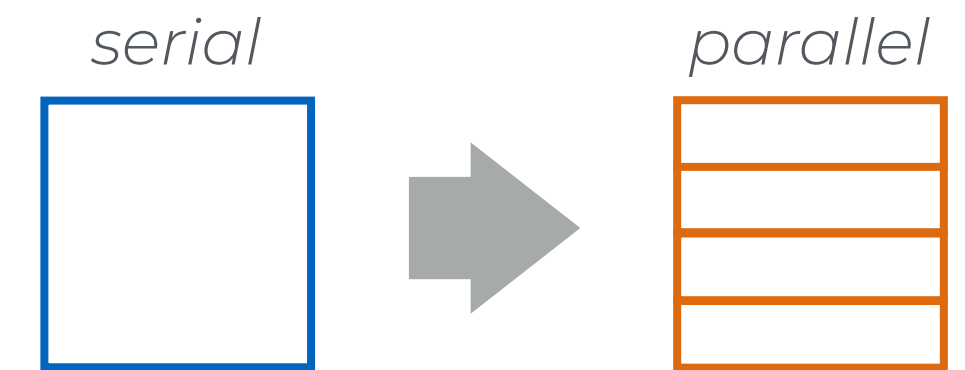
# Scaling laws



# Scaling on HPC systems

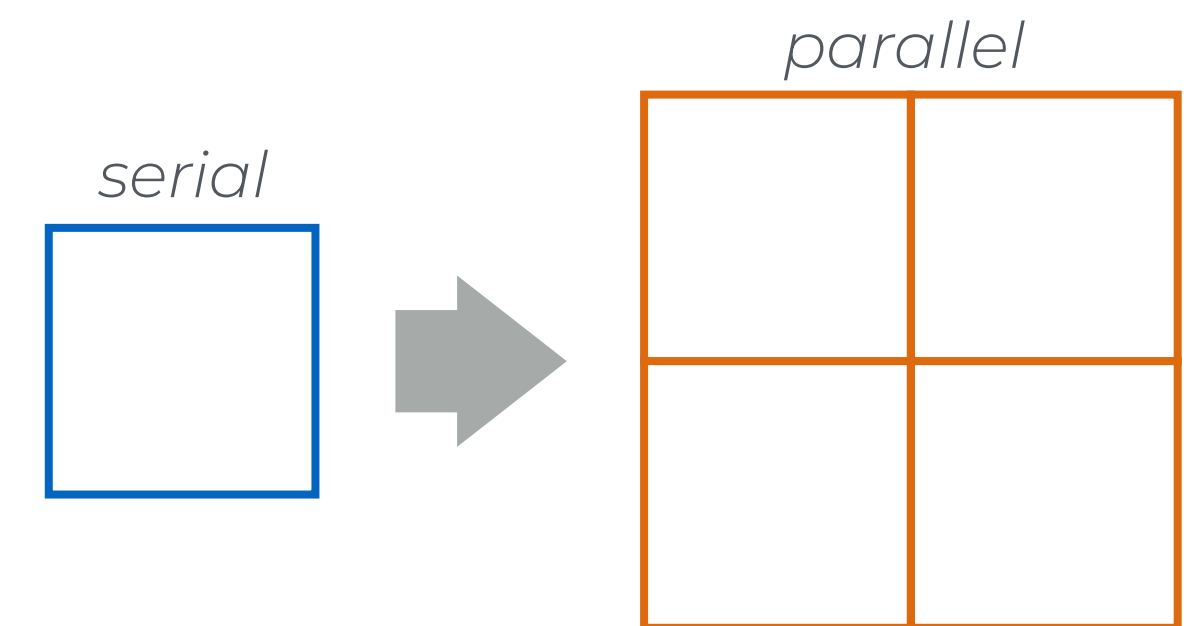
- **The use of HPC systems allows us to overcome the limits of single CPU computing**
  - We can solve problems that can't be solved in a reasonable time and/or (capability)
  - We can solve problems that don't fit on a single processor (capacity)
- **This leads to two types of scalability analysis**
  - Strong scaling and weak scaling
- **Strong scaling focuses on fixed problem sizes**
  - It measures how much faster we can run a given problem when we increase the number of processors
- **Weak scaling focuses on increased problem size proportional to processors**
  - The bigger problem in parallel should take the same runtime as the smaller problem on a single processor

## strong scaling



*same problem faster*

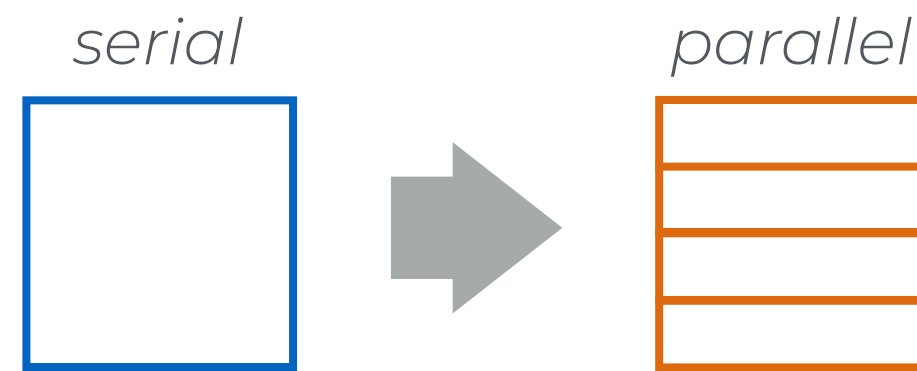
## weak scaling



*larger problem, same time*

# Strong and weak scaling

## strong scaling



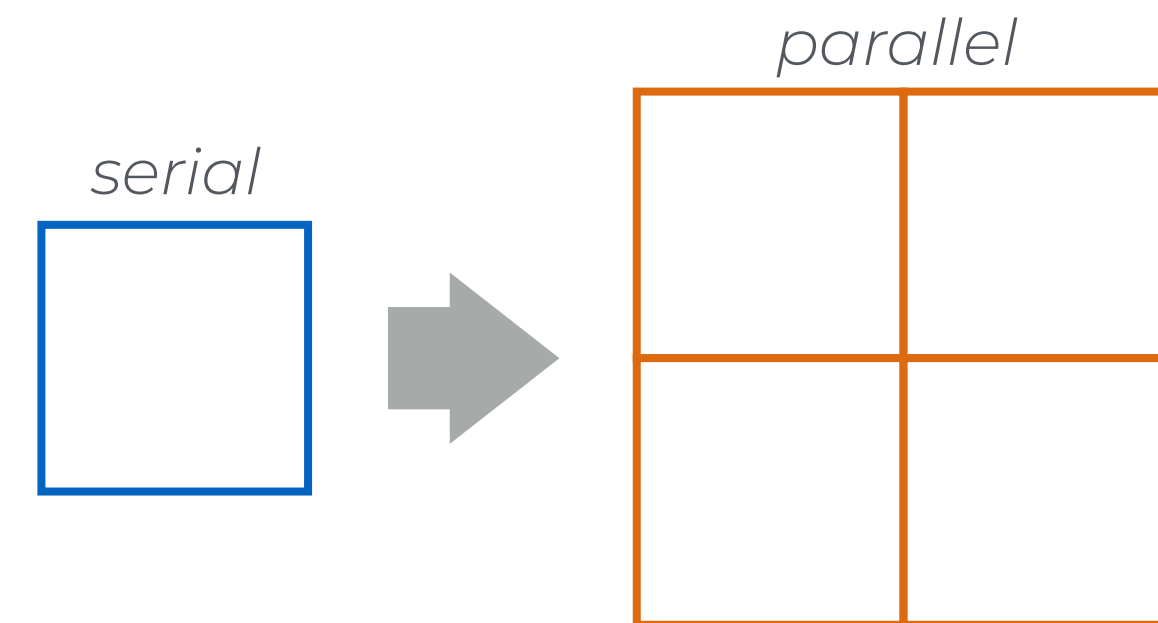
- **Fixed-size problems**

- Increase the number of processing elements
- Ideal runtime  $T_p = T_1/p$
- Strong scaling efficiency  $E_p = \frac{T_1}{p T_p}$

- **Why strong scale breaks down**

- As  $p$  increases, each processor gets less work. Eventually, overhead dominates.
- Common limiting factors: serial sections, latency, global reductions, Load imbalance

## weak scaling



- **Work per processor constant**

- Increase problem size with number of processors
- Ideal result  $T_p = T_1$
- Weak scaling efficiency  $E_p = \frac{T_1}{T_p}$

- **Unfortunately, weak scaling also degrades**

- Increasing global communication, longer communication paths, I/O bottlenecks
- Load imbalance, Network congestion, Algorithmic effects



# Amdahl's Law

- **The simplest model for strong scaling (fixed problem size) is Amdahl's law**

- It considers only the effect of having a serial fraction in the code
- This fraction will not benefit from parallelization

- **Amdahl's law puts a very stringent limit on speedup**

- Parallel runtime will be given by

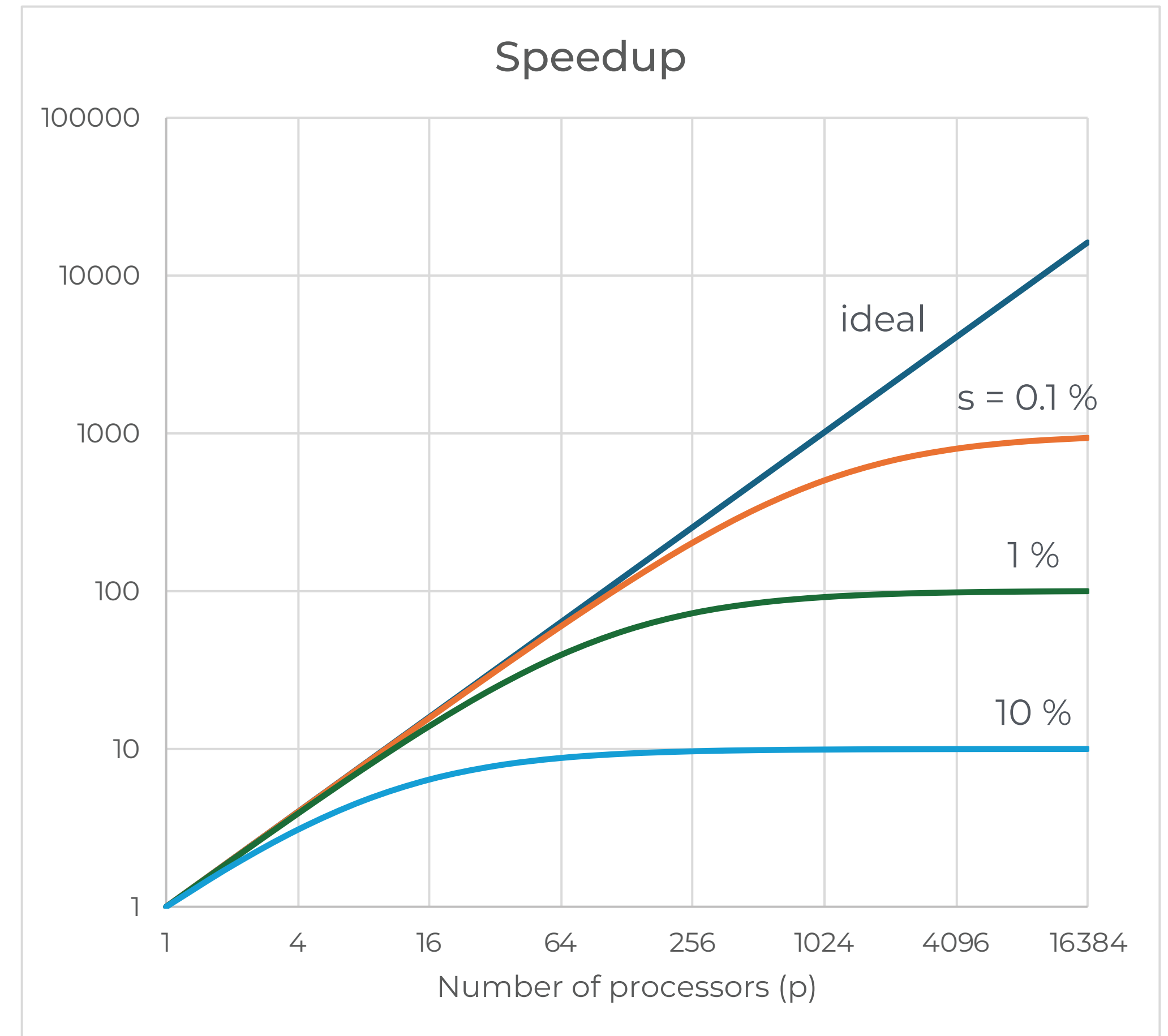
$$T_p = \left( s + \frac{1-s}{p} \right) T_1$$

- Where  $s$  is the serial fraction of the code,  $1 - s$  is the parallel fraction and  $p$  is the number of processors
- The maximum possible speedup would then be

$$S_p = \frac{1}{s + \frac{1-s}{p}}$$

- **As  $p \rightarrow \infty$  the maximum speedup is  $S_\infty = \frac{1}{s}$**

- A small serial fraction can severely limit strong scaling



**strong scaling**

# Gustafson's Law

- **Gustafson's Law models scaled speedup.**

- Instead of fixing the problem size, we increase the problem size as we increase p.

- **Speedup is calculated in comparison with a serial system running the scale problem**

- In this case the runtime is expected to be

$$T_1 = (s + p(1 - s)) T_0$$

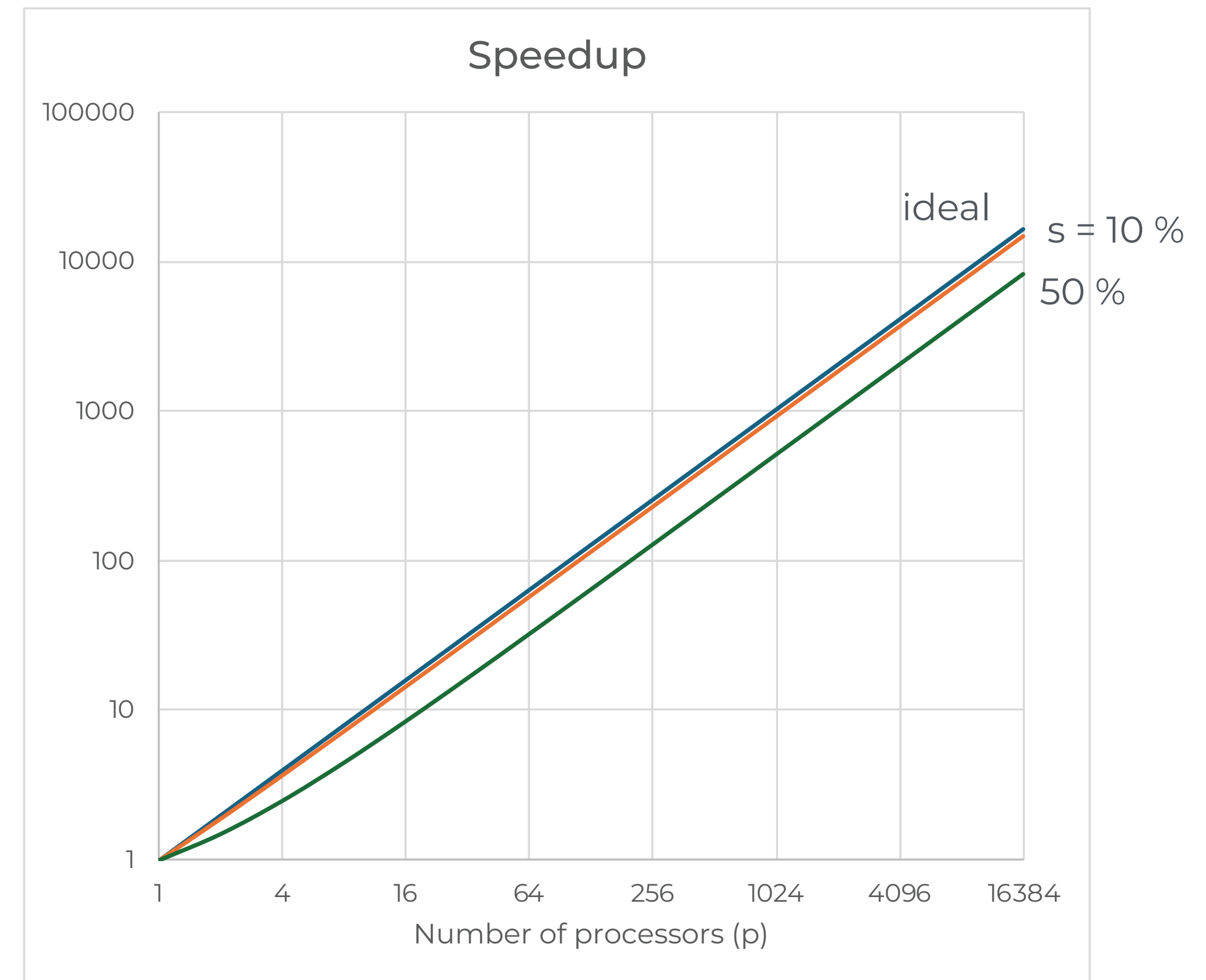
- Where  $T_0$  is the time it takes to run the small problem serially, and  $p(1 - s)$  represents the scaled problem

- **In this case speedup is then found to be**

$$S_p = s + p(1 - s)$$

- **More processors also allow solving larger problems in the same time**

- Ahmdal's law is often pessimistic; users frequently increase problem size when more resources are available



**weak scaling**

# Example: 2D stencil

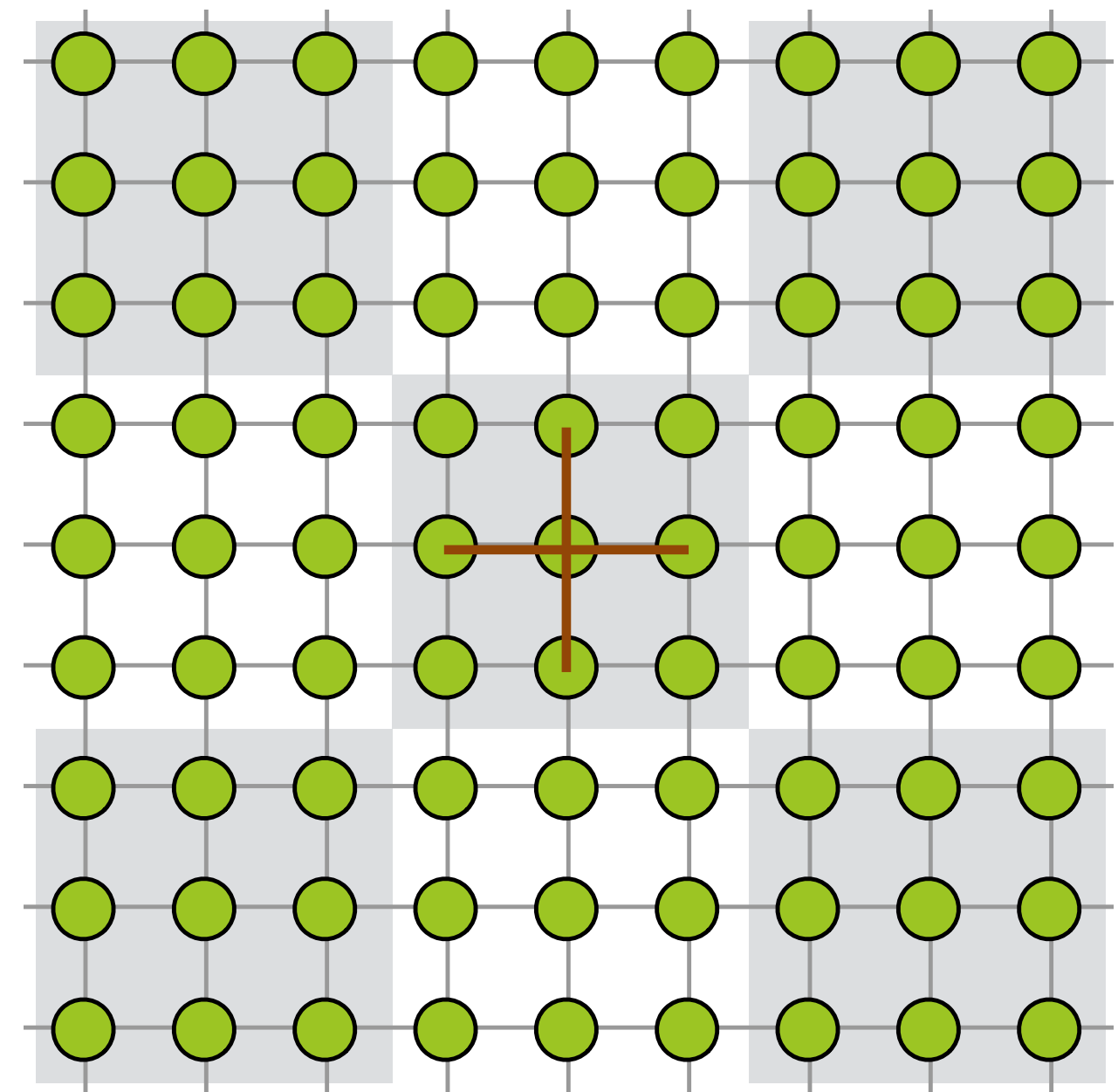


# 2D stencil problem

- **Consider a problem using an  $N \times N$  grid**
  - Each point update uses only values from the neighboring points
- **Assume each point update takes a constant number of operations**
  - For simplicity we assume that this corresponds to a constant cost  $c$
  - We do not account for cache or memory bandwidth bottlenecks
- **The problem will be partitioned across  $p$  processors**
  - More specifically, we will partition the domain in 2 dimensions
  - The grid size per process will be  $\frac{N}{\sqrt{p}} \times \frac{N}{\sqrt{p}} = \frac{N^2}{p}$
- **A simple estimation of the compute time then yields**

$$T_{\text{compute}} = c \frac{N^2}{p}$$

- Since the grid is uniform and decomposition is uniform we also expect  $T_{\text{imbalance}} \simeq 0$

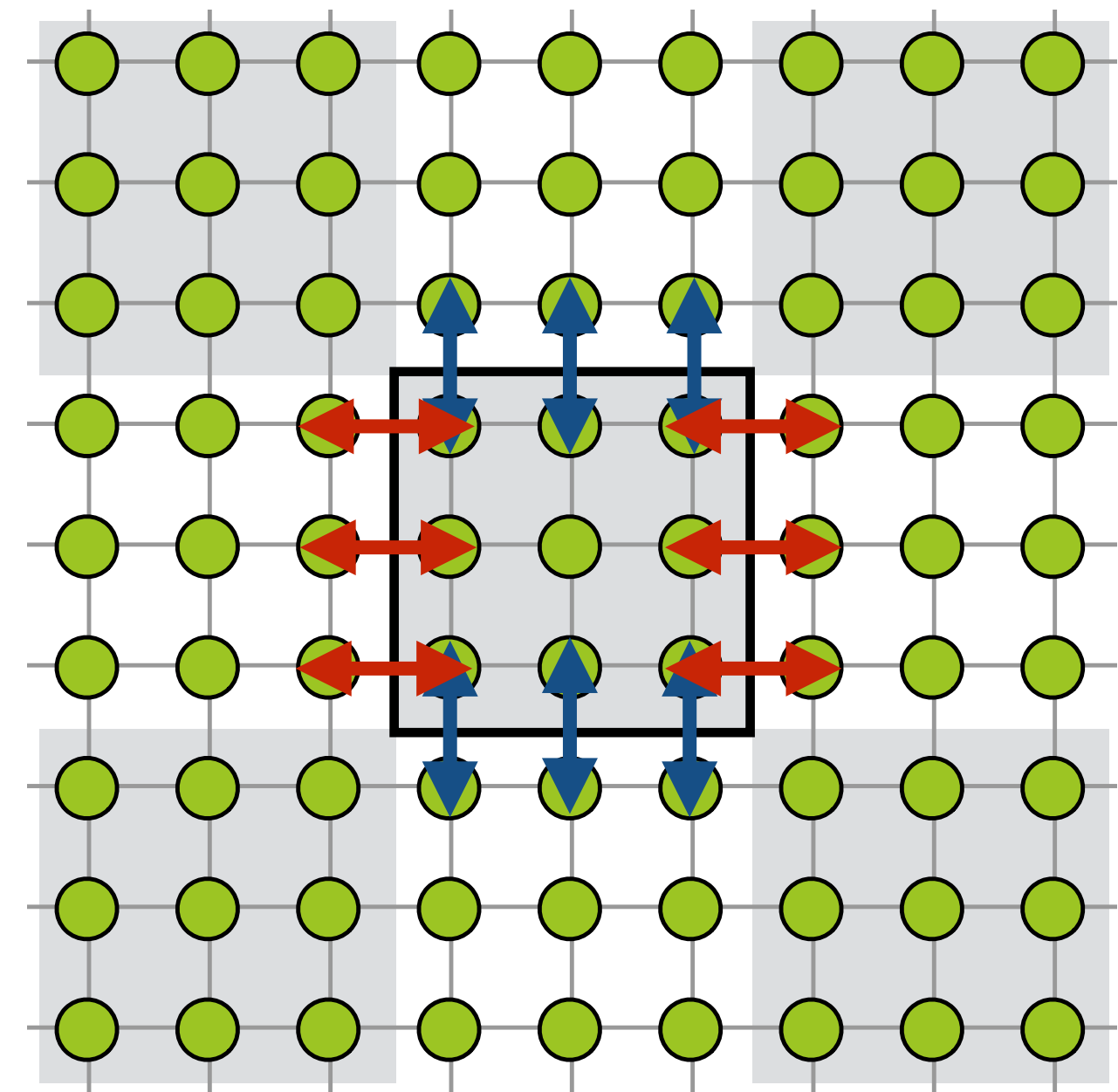


# Communication cost

- **After each grid update the code must update guard cell values**
  - This is also referred to as halo exchange
- **Each process exchanges guard rows/columns with four neighbors**
  - Up, down, left, right
  - Given that the grid size per process is  $\frac{N}{\sqrt{p}} \times \frac{N}{\sqrt{p}}$
  - Each boundary length is then  $N/\sqrt{p}$
- **We use the latency-bandwidth model to estimate communication cost**
  - Assuming that  $k$  is the number of bytes per boundary element we have:

$$T_{\text{comm}} = 4 \left( L + \frac{k}{B_{\text{net}}} \frac{N}{\sqrt{p}} \right)$$

- Where again  $L$  is the latency and  $B_{\text{net}}$  is the bandwidth. Each process exchanges 4 messages.





# Total runtime model

- Using the assumptions above, we get a simple model for runtime per iteration

$$T_p = c \frac{N^2}{p} + 4 \left( L + \frac{k}{B_{\text{net}}} \frac{N}{\sqrt{p}} \right)$$

- We can now use this model to estimate strong and weak scaling

- **Strong scaling**

- For fixed  $N$  we see that  $T_{\text{compute}} = \mathcal{O}\left(\frac{N^2}{p}\right)$  and  $T_{\text{comm}} = \mathcal{O}(1) + \mathcal{O}\left(\frac{N}{\sqrt{p}}\right)$  (latency + bandwidth)

- As  $p$  increases computation decreases rapidly. However communication decreases more slowly and latency does not decrease
- Eventually communication and latency dominate and strong scaling saturates

- **Weak scaling**

- For  $N^2/p$  constant, that is,  $N \propto \sqrt{P}$ , compute time is constant
- Communication time is also constant
- Good weak scaling is to be expected (as long as only point to point communications are used)

# Overview

# Overview

- **Performance modeling helps us understand why parallel programs scale, or why they fail to scale**
  - The goal is not to predict runtimes perfectly but rather to identify dominant costs and explain scaling behaviors
- **The main parallel performance metrics are speedup, efficiency and cost**
  - Our performance model will helps improve on these values and/or identify the major bottlenecks
- **Performance models help use establish performance bounds for our program**
  - Roofline and latency-bandwidth models help us understand the basic compute and network performance of our code
- **Parallel scalability tells us how well our code is benefitting from larger numbers of processors**
  - HPC systems will allow us to run the same problems faster (strong scaling) or larger problems in the same time (weak scaling)

| Model             | Primary Focus      | Best Used For...                                             |
|-------------------|--------------------|--------------------------------------------------------------|
| Amdahl            | Serial bottlenecks | Finding the "wall" of an algorithm.                          |
| Gustafson         | Scaled performance | Analyzing large scale problems                               |
| Latency-Bandwidth | Network overhead   | Optimizing MPI communication patterns.                       |
| Roofline          | Hardware limits    | Identifying if you should optimize kernels or memory access. |